TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE 4302 — Arquitectura de Computadores II

# Proyecto Individual
## Framework experimental de modelos de ejecución **Multithreading**

| | | | |
|---|---|---|---|
| Fecha de asignación: | 24 de febrero 2026 | Fecha de entrega: | 26 de marzo 2026 |
| Grupos: | 1 persona | Profesores: | Ronald García Fernández<br>Luis Chavarría Zamora |

## 1. Objetivo

Mediante el desarrollo de este proyecto, la persona estudiante aplicará conceptos de paralelismo a nivel de hilo, modelos de ejecución concurrente y evaluación de desempeño en sistemas multinúcleo. El objetivo principal consiste en diseñar e implementar un problema altamente paralelizable y evaluar su comportamiento bajo distintos modelos de ejecución que aproximen mecanismos utilizados en arquitecturas modernas.

la persona estudiante deberá comprender las diferencias entre modelos de ejecución a nivel de software y mecanismos soportados directamente por hardware, así como las limitaciones prácticas de su implementación en computadores de propósito general.

## 2. Descripción general

Los sistemas computacionales modernos utilizan múltiples niveles de paralelismo para incrementar el desempeño, incluyendo paralelismo a nivel de hilo, ejecución simultánea y multiprocesamiento multinúcleo. Sin embargo, muchos de estos mecanismos corresponden a decisiones microarquitectónicas que no siempre pueden implementarse directamente desde el software de aplicación.

En este proyecto se diseñará un sistema experimental que permita comparar distintos modelos de ejecución concurrente mediante implementación directa, simulación o aproximación experimental. La persona estudiante deberá desarrollar una plataforma de pruebas que permita ejecutar un problema altamente paralelizable bajo distintos esquemas de ejecución, recolectar métricas de desempeño y analizar las diferencias observadas con base en la teoría de arquitectura de computadores y paralelismo.

El enfoque del proyecto es experimental y comparativo, enfatizando la medición rigurosa y el análisis crítico de resultados.

# 3. Especificación

En este proyecto la persona estudiante debe proponer un problema altamente paralelizable y evaluar su comportamiento bajo distintos modelos de ejecución concurrente. Dependiendo del modelo, la implementación podrá ser directa, simulada o aproximada experimentalmente.

1. El sistema debe ejecutarse bajo los siguientes modelos (aplicando simulación o aproximación cuando el mecanismo sea microarquitectónico):

   *a*) Multihilo de grano fino (**Fine-Grained Multithreading**): Debe implementarse mediante un modelo de planificación cooperativa o simulación de ejecución por ciclos o cuantums, donde el cambio de hilo ocurre en cada ciclo (o cuantum mínimo) en forma *round-robin*, independientemente de si el hilo actual ha sufrido un *stall*.

   *b*) Multihilo de grano grueso (**Coarse-Grained Multithreading**): Implementación directa mediante hilos tradicionales donde el cambio de contexto ocurre únicamente ante eventos de bloqueo costosos (sincronización, fallo de caché, espera o E/S).

   *c*) Multihilo simultáneo (**SMT – Simultaneous Multithreading**): Debe aproximarse mediante ejecución concurrente con sobre-suscripción de hilos respecto a los núcleos físicos o mediante simulación de unidades funcionales compartidas. **Nota importante:** Para aislar el efecto de SMT en las mediciones, la persona estudiante debe deshabilitar SMT (Hyper-Threading en Intel, SMT en AMD) desde la configuración del BIOS/UEFI del sistema de pruebas y comparar los resultados contra la ejecución con SMT habilitado. Esto permite cuantificar el impacto real del multihilo simultáneo por hardware versus la aproximación por software.

   *d*) Multiprocesamiento multinúcleo (**CMP – Chip Multiprocessing**): Implementación directa usando paralelismo real sobre múltiples núcleos mediante procesos o hilos del sistema operativo.

2. La persona estudiante puede utilizar cualquier lenguaje de programación. Debe justificar las decisiones de implementación considerando el modelo de concurrencia del lenguaje, sus mecanismos de sincronización y sus limitaciones respecto al paralelismo real. No se permite el uso de lenguajes de alto nivel de abstracción como Python o C#.

3. El problema es propuesto por la persona estudiante. Se recomienda analizar suites de benchmarks paralelizables para inspiración conceptual. A continuación se sugieren algunos temas relevantes que presentan alto grado de paralelismo:

   - **Procesamiento de Big Data**: operaciones masivas sobre bases de datos no relacionales (NoSQL), por ejemplo, consultas paralelas sobre MongoDB, agregaciones distribuidas o procesamiento *map-reduce* sobre grandes volúmenes de datos.

- **Simulación de partículas**: simulaciones N-body, dinámica molecular o sistemas de partículas donde las interacciones pueden particionarse espacialmente para su ejecución concurrente.
- **Ray tracing**: renderizado de escenas 3D mediante trazado de rayos, donde cada rayo o región de la imagen puede procesarse de forma independiente, siendo un problema inherentemente paralelizable.

4. Para cuantificar la mejora deben proveer al menos las siguientes métricas:

   *a*) Tiempo total de ejecución.

   *b*) Speedup respecto a la versión secuencial.

   *c*) Eficiencia paralela.

   *d*) Escalabilidad en función del número de hilos o procesos.

   Adicionalmente, la persona estudiante deberá utilizar al menos una herramienta de perfilado que permita analizar de forma explícita el comportamiento bajo SMT (hilos lógicos), tales como:

   - `perf` en Linux, usando contadores de hardware y eventos relacionados con SMT. Véase la documentación oficial en https://perfwiki.github.io/main/.
   - Intel® VTune Profiler, para análisis detallado de *threading*, microarquitectura y uso de recursos. Disponible en https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html.

   Se deben incluir en el documento capturas o resúmenes de las métricas relevantes obtenidas con estas herramientas (por ejemplo, ciclos, instrucciones, uso de CPU por hilo lógico, *stalls*, etc.), discutiendo cómo se relacionan con los distintos modelos de ejecución (fine-grained, coarse-grained, SMT, CMP). **Sugerencia**: Implemente mediante *scripts* la generación de datos y gráficas de manera automatizada.

   Por razones de validez experimental, **no se permite realizar las mediciones principales en entornos virtualizados** (máquinas virtuales, contenedores, WSL u otros). Las pruebas deben ejecutarse directamente sobre hardware físico dedicado, ya que la virtualización introduce sobrecarga, contención de recursos y variabilidad que distorsionan los resultados de benchmarking y perfilado. la persona estudiante deberá analizar críticamente posibles sesgos experimentales y validar la coherencia de sus resultados con la teoría.

El sistema debe ser probado exhaustivamente al menos 200 ejecuciones en cada caso (dependiendo del problema, puede requerir más o menos ejecuciones en función de sus variables), esto para cuantificar la mejora. Se sugiere implementar *scripting* (e.g., `Makefile`). El sistema debe

generar gráficas de las métricas, esto facilitará la interpretación. Usen como base el problema sin esquema de hilos.

**Justificación de pruebas repetidas:** la persona estudiante debe justificar explícitamente el número de iteraciones realizadas para cada configuración (al menos 200 ejecuciones mínimas como referencia, pero adaptadas al problema y tamaño de entrada). Se requiere:

- Cálculo del intervalo de confianza (por ejemplo, 95 %) de las métricas principales (tiempo, speedup) usando estadística descriptiva (media, desviación estándar).
- Gráficas de boxplot o histograma mostrando la distribución de los resultados de todas las iteraciones por configuración (hilos, modelo de ejecución).
- Discusión sobre la convergencia de los resultados, identificando si se necesita mayor número de iteraciones para estabilidad.

Esto debe documentarse en la sección de resultados del paper, incluyendo código o script utilizado para las repeticiones (e.g., bash loop, Makefile target). Las gráficas deben generarse automáticamente desde los datos recolectados.

## 3.1. Notas adicionales

1. El desarrollo de este proyecto es individual.
2. Deben usar Github, este debe ser privado, usen la Metodología indicada en la sección 4.
3. El proyecto tendrá dos demostraciones o evaluaciones antes de la entrega final, estos serán los requerimientos mínimos de los avances:

   *a*) Demostración 1: Martes 3 de marzo. la persona estudiante debe mostrarle al profesor el problema escogido, las razones para fundamentar su selección. Aquí deberá proponer mediante diagramas (con base en la teoría) sobre cómo implementará el multihilo de grano fino (*fine-grained*) y el multihilo de grano grueso (*coarse-grained*).

   *b*) Demostración 2: Martes 17 de marzo. la persona estudiante debe mostrarle al profesor los diagramas de las soluciones esquematizadas preliminares (indicando los cambios más significativos). Ejecución del sistema base (sin hilos). Identificación de las variables más importantes del problema que aumenten su paralelización. Una ejecución *dummy* o trivial de cada esquema multihilo con las mediciones, puede ser en partes parciales del problema.

   Estas demostraciones son obligatorias pues permiten tener certeza que el producto final llegue con menos fallas.

4. Todo diseño deberá tener al menos 2 propuestas detalladas adecuadamente y comparadas según criterios.

# 4. Metodología de trabajo

El proyecto debe seguir los siguientes aspectos de desarrollo, sino, la parte funcional no será calificada y obtendrá nota de cero. la persona estudiante deberá seguir los siguientes lineamientos:

1. Los estudiantes pueden usar herramientas de inteligencia artificial u otras fuentes para guía. Debe documentar los procedimientos automatizados y los mismos deben ser validados por usted contra referencias verificadas y confiables [1]. **En este proyecto usted será un diseñador(a), no un copiloto(a) sin criterio técnico crítico. Esto debe formar parte de los anexos en el documento entregado.**

2. Todo código debe ser documentado de forma interna.

3. Manejo por repositorio:

   *a*) Utilice una cuenta de repositorio gratuita.

   *b*) El repositorio debe ser enviado al profesor no más de 5 hábiles después de la entrega de este enunciado. Sino, habrá **penalización** de un punto por hora en la nota final del proyecto después de las 11:59 pm del quinto día hábil.

   *c*) Cree un repositorio con el siguiente nombre: `<user_id>_a2_2026_s1`. El `user_id` estará compuesto por la primera letra del nombre y el apellido. Por ejemplo, para la persona estudiante Fulanito Pérez, el nombre del repositorio será: `fperez_a2_2026_s1`.

   *d*) El repositorio debe ser mediante el uso de GitHub, debe ser privado, proporcione acceso a `luchazam` (Grupo 01) o `rgarciatec` (Grupo 02). Si no es privado desde el inicio recibirá penalización de 10 puntos.

   *e*) El repositorio de Git contendrá dos ramas principales: `master` y `development`.

   *f*) Inicialmente, la rama de `development` se crea a partir del `master`.

   *g*) Al trabajar en un proyecto, la persona estudiante debe crear una nueva rama de trabajo desde `develop` y cuando la función esté lista, la rama debe fusionarse para `develop`. Cualquier corrección o modificación adicional después de `merge` debería requerir que se repita el proceso (es decir, crear la rama desde `develop` y fusionar los cambios más tarde). Una vez que el código de desarrollo esté listo, se fusionará con `master` y se debe crear una `tag`. El proceso se describe en la Figura 1.

[1] Algunas veces estas herramientas inducen a sesgos y no deben ser tomadas sin cuestionamiento o verificación (como todo en ingeniería).

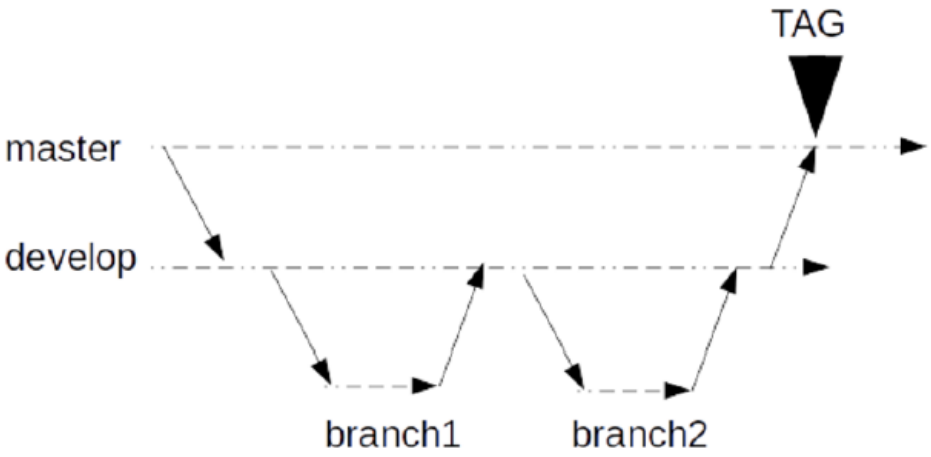



Figura 1: *Git workflow*

Adicionalmente se coloca este enlace recomendado.

*h*) Solo debe contemplar código, recuerde que es un repositorio de versiones de código, no un Drive.

*i*) Después de haber realizado algunos proyectos la rama `master` debe verse así:

- `master/`
  - `proyecto_1`
  - `proyecto_2`
  - ...
- ...

Donde cada directorio de `proyecto_x` contiene todos los entregables para cada proyecto.

**No es permitido realizar todo el trabajo en un solo commit, es decir, que realice el trabajo de forma local y solo suba el último entregable en el repositorio. Si no, será penalizado en todos los rubros de evaluación de la defensa disminuyendo un escalafón en la calificación de cada rubro. Debe mostrar avance incremental (se revisarán estadísticas).**

# 5. Entregables

Como entregables en este proyecto se evaluará lo siguiente:

- Presentación funcional completa de **30 minutos** (75 %): Cada estudiante deberá demostrar en una sesión (previa cita con el profesor) las diferentes pruebas sobre el sistema. El profesor evaluará las pruebas según rúbrica correspondiente, se realizarán preguntas orales de lo implementado (todo debe ser fundamentado, sino habrá penalidad conceptual). En la sesión se harán preguntas relacionadas sobre cualquier etapa del sistema, se contrastará contra la . El día de la presentación, la persona estudiante cargará el último commit antes de la primera revisión.

- Artículo tipo *paper* en formato PDF (25 %): El artículo deberá tener una extensión no mayor a 4 páginas y debe seguir el formato IEEE Transactions. Se recomienda el uso de LaTeX para su elaboración, sin embargo, no es obligatorio; la persona estudiante puede utilizar cualquier herramienta de edición siempre que el documento final se entregue en formato PDF y respete el formato IEEE indicado. En general el *paper* deberá contar con las siguientes secciones:

1. Abstract (en inglés): Un buen abstract tiene las siguientes características:

   *a*) Un abstract permite a los lectores obtener la esencia o esencia de su artículo o artículo rápidamente, para decidir si leer el artículo completo.

   *b*) Un abstract prepara a los lectores para seguir la información detallada, los análisis y los argumentos en su artículo completo.

   *c*) Un abstract ayuda a los lectores a recordar puntos clave de su pap.

   *d*) Un abstract es de entre 150 y 250 palabras.

2. Introducción: Una buena introducción muestra el contexto del problema o lo que se va a solucionar, introduce el tema al lector. Al final de la introducción se indica la organización del documento (primero se muestra la investigación, luego....).

3. Marco teórico.

4. Sistema desarrollado con opciones de solución.

5. Resultados y análisis con respecto a la teoría.

6. Conclusiones escritas en prosa.

7. Bibliografía, en formato IEEE y referenciadas en el texto (usar cite). Referencia bien para evitar problemas de plagio.

Si tienen dudas puede escribir al profesor al correo electrónico de Luis Chavarría, correo electrónico de Ronald García. **Los documentos serán sometidos a control de plagios para eliminar cualquier intento de plagio con trabajos de semestres anteriores, actual o**

**copias textuales, tendrán nota de cero los casos detectados**. La entrega se debe realizar por medio de TEC-Digital en la pestaña de evaluación a más tardar las 11:59 pm, no se aceptan entregas extratemporales. Los documentos serán sometidos a control de plagios. **Se prohíbe el uso de referencias hacia sitios no confiables.**